



KOM120C **PEMROGRAMAN**

K11 | Pekan Ke-12 **Data Structure and Container**

Array and List

Tim Pengajar Pemrograman IPB University

Learning Outcome

- Mahasiswa memahami pengertian struktur data dan container.
- Mahasiswa memahami struktur data Array.
- Mahasiswa memahami container List dan Set
- Mahasiswa mampu membuat program menggunakan struktur data Array dan container List dan Set.



Struktur Data & Container

- **Struktur data** adalah tools yang sangat penting dalam ilmu komputer dan pemrograman yang mengatur dan menyimpan data secara efisien. Struktur data menentukan cara data disimpan, diakses, dan dimanipulasi. Contoh struktur data: **Arrays, Linked Lists, Stacks and Queues, Hash Tables, Trees and Graphs.**
- **Container** merupakan **struktur data abstrak** yang disediakan oleh pustaka pemrograman untuk menyimpan dan mengatur data. Container sering dibangun menggunakan struktur data dasar tetapi dilengkapi dengan berbagai fungsi tambahan. Contoh:
 - Lists, Sets, Maps dalam Java Collection Framework.
 - Vectors, Deques, Unordered Maps dalam C++ Standard Template Library.
 - Dictionaries, Sets, Tuples dalam Python
 - Lists, Sets, Maps, Tuples dalam Scala

Manfaat

- Singkatnya, struktur data adalah pondasinya, container adalah implementasi yang membuatnya lebih mudah diakses dan praktis.
- Setiap struktur data/container memiliki tujuan dan kegunaan masing-masing. Jika kita menggunakannya sesuai untuk tujuan tertentu, maka program kita akan menjadi lebih efisien, "bersih", dan mudah untuk di-maintain.
 - **Efisiensi Operasi.** Misalkan hashtable memungkinkan pencarian, penambahan, dan penghapusan data dilakukan secara cepat
 - **Pengorganisasian Data.** penyimpanan data secara terorganisir berdasarkan kebutuhan spesifik seperti Tree dan Graph.
 - **Keterbacaan dan Pemeliharaan Kode:** kode menjadi lebih bersih, mudah dibaca, dan lebih mudah dipelihara, karena fungsi-fungsi dasar sudah tersedia.
 - **Skalabilitas:** struktur data yang efisien memungkinkan aplikasi mengelola data dalam jumlah besar tanpa kehilangan kinerja.

Array

- Struktur data paling sederhana adalah Array.
- Array pada Scala adalah struktur data yang memiliki sifat:
 - Berukuran tetap
 - Mutable (dapat diubah)
 - Setiap elemen bertipe sama
 - Dapat diakses menggunakan indeks

```
val nums = Array(1,2,3,4,5)
val strs = Array("Scala", "Java", "Phyton")
val zeros = Array.fill(5)(0)

nums.foreach(print)
println()
println(zeros.mkString(" "))
```



```
12345
0 0 0 0 0
```

Operasi Array

- Akses elemen array menggunakan tanda kurung bisa. Indeks dimulai dari 0 (0-Based).

```
val nums = Array(1,2,3,4,5)
val strs = Array("Scala", "Java", "Phyton")
println(nums(2))
println(strs(1))

nums(3) = 10
println(nums.mkString(" "))
```



```
3
Java
1 2 3 10 5
```

- Kita dapat menerapkan berbagai HOF untuk mengolah array.

```
println(strs.length)
println(nums.map(_*2).mkString(" "))
println(nums.filter(_%2==0).mkString(" "))
println(strs.filter(_.length<=5).mkString(" "))
println(nums.sorted.mkString(" "))
println(nums.reduce(_+_))
```



```
3
2 4 6 20 10
2 10
Scala Java
1 2 3 5 10
21
```

Contoh #1

Problem

Buatlah sebuah program Scala yang menerima input berupa sebuah Array, kemudian menghasilkan Array baru yang elemen-elemennya merupakan kuadrat dari semua elemen-elemen yang ada pada Array input yang ganjil, diurutkan dari elemen terkecil ke terbesar!

Ide Solusi

Input fungsi berupa Array bilangan bulat, misalkan Arr → filter elemen ganjil → urutkan (sorted) dari kecil ke besar → kuadratkan masing-masing elemen.

Scala

```
def arrKuadrat(arr: Array[Int]): Array[Int] = {  
    arr.filter(_%2!=0).sorted.map(x=>x*x)  
}  
  
def main(args: Array[String]): Unit = {  
    val nums = Array(3, 2, 5, 6, 8, 2, 1)  
    val result = arrKuadrat(nums)  
    println(result.mkString(" "))  
}
```

→ 1 9 25

List

- List adalah kumpulan koleksi data/nilai yang tersusun secara linear dan bersifat immutable (setelah dibuat, maka nilainya tidak dapat diubah lagi).
- Seperti halnya Array, semua elemen pada sebuah List harus memiliki tipe yang sama.
- List dapat diinisialisasi dengan nilai-nilai yang diinginkan. Setelah List dibuat, kita tidak dapat mengubah nilai elemennya.
- List kosong dinyatakan dengan kata kunci Nil.

```
val nums = List(1,2,3,4,5)
val strs = List("Scala", "Java", "Phyton")
val emptyList1 = List()
val emptyList2 = Nil

println(nums(2))
println(strs.mkString(" "))
println(emptyList2)

nums(2)=10    // ERROR
```



```
3
Scala Java Phyton
List()
```


Mengolah List

Meskipun kita tidak dapat mengubah isi elemen dari sebuah List, kita dapat melakukan operasi untuk menambah elemen List atau menggabungkan dua buah List menjadi satu.

```
val bil1 = List(1,2,3)
val bil2 = List(4,5)

val nums1 = bil1 ++ bil2
val nums2 = bil1.concat(bil2)
val nums3 = bil1 :+ 9
val nums4 = bil1 :+ bil2

println(nums1.mkString(" "))
println(nums2.mkString(" "))
println(nums3.mkString(" "))
println(nums4.mkString(" "))
```



```
1 2 3 4 5
1 2 3 4 5
1 2 3 9
1 2 3 List(4, 5)
```

Operator :: pada List

- Secara teknis, List diimplementasikan sebagai linked list, yang memudahkan operasi untuk mendapatkan elemen pertama (head) atau elemen terakhir (tail). Pada Scala digunakan operator :: seperti pada contoh berikut:

```
val head::tail = List(5, 3, 1, 4, 2)
println(head)
println(tail)
```



5
List(3, 1, 4, 2)

- Pemecahan List menjadi head dan tail banyak digunakan untuk mengolah List secara rekursif. Contoh: menghitung penjumlahan semua elemen List.

```
def jumlah(L: List[Int]): Int = {
  L match {
    case Nil => 0
    case head::tail => head + jumlah(tail)
  }
}
def main(args: Array[String]): Unit = {
  val nums = List(1,2,3,4,5)
  println(jumlah(nums))
}
```

Bandingkan dengan HOF:
L.reduce(_+_)



15

Contoh #2

Problem

Buatlah program menghitung berapa banyak kemunculan sebuah nilai tertentu dalam sebuah List. Contoh: jika data = List(1, 2, 3, 1, 4, 3).

Ide Solusi

Sangat mudah menggunakan HOF (**count**). Namun dapat pula menggunakan fungsi rekursif (diperlukan saat tidak dapat diselesaikan dengan HOF), yaitu memeriksa satu persatu secara linear, mulai dari head sampai habis.

Scala

```
def linearSearch1(L: List[Int], q: Int): Int = {           // HOF
  L.count(_==q)
}

def linearSearch2(L: List[Int], q: Int): Int = {           // Rekursif
  L match {
    case Nil => 0
    case head::tail =>
      if (head==q) 1+linearSearch2(tail,q)
      else linearSearch2(tail,q)
  }
}
```

Contoh #3

Problem

Buatlah program menghitung jumlah dari semua elemen dari sebuah List yang genap.

Ide Solusi

Sangat mudah menggunakan HOF (**filter** yang genap, kemudian **reduce**). Namun secara proses, dapat dibuat fungsi rekursif, yaitu memeriksa satu persatu secara linear kemudian menjumlahkan jika elemennya genap.

Scala

```
def sumGenap1(L: List[Int]): Int = {  
  L.filter(_%2==0).reduce(_+_)  
}  
  
def sumGenap2(L: List[Int]): Int = {  
  L match {  
    case Nil => 0  
    case head::tail =>  
      if (head%2==0) head+sumGenap2(tail)  
      else sumGenap2(tail)  
  }  
}
```

Diskusi Kelas #1

Buatlah program untuk melakukan proses merge sort dari 2 list yang elemennya sudah terurut, menghasilkan sebuah list baru yang terurut.

Contoh Input:

3

1 4 7

4

3 9 10 15

Contoh Output:

1 3 4 7 9 10 15

Diskusi Kelas #2

Buatlah program Scala untuk menghapus elemen suatu list yang sama dan bersebelahan.

Contoh Input:

11

1 1 1 4 4 1 1 1 1 1 7

Contoh Output:

1 4 1 7

Diskusi Kelas #3

Buatlah program Scala untuk menghapus (semua) elemen suatu list yang sama dan bersebelahan.

Contoh Input:

11

1 1 1 4 4 1 1 1 1 1 7

Contoh Output:

7